

Agenda

Fundamental Terms and Concepts

Installing and setting up python

Literals, Variables, and Numeral Systems

Input and Output in Python (Console I/O)

Operators and Expressions

timings

9.30AM to 11.00AM – P1

15 min Break

11.15 to 1.30PM – P2

1 Hr Break

2.30 to 3.45 – P3

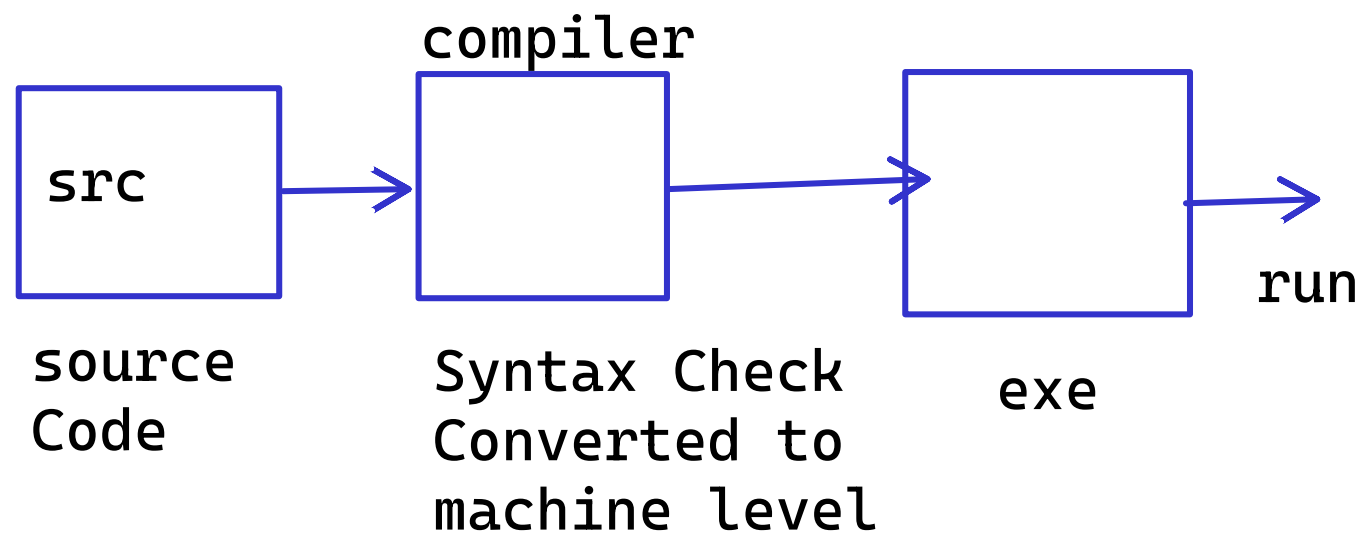
15 Break

4.00PM to 5.30PM – p4

Programming Languages

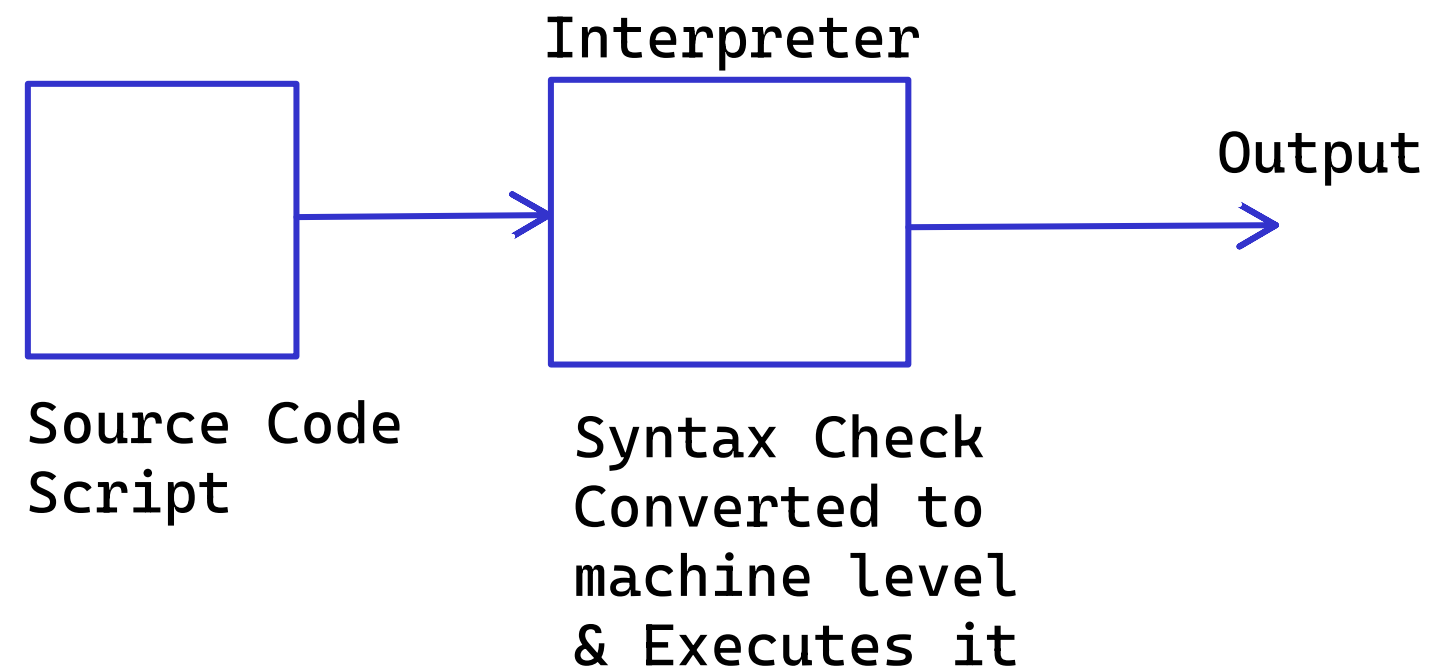
- Digital 1/0
- High Level instructions to Machine level Code (1/0)

Compiled Languages



Eg: C/C++/C#/Go/Rust

Interpreted Languages



Command by command

Eg: Python/JavaScript.....

Compiled Language

- More Faster
- Granular Control
- Takes hardware capability
- Secure compared to interpreted
- user does not need any additional software to use
- Takes time to compile
- Advanced/ Expertise needed to develop secure app

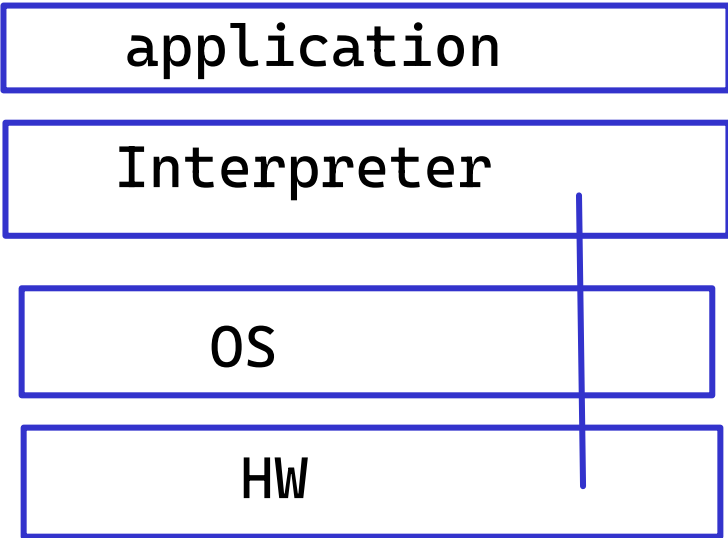
Interpreted Language

- More beginner Friendly
- less control compared to compiled
- slower than compiled
- Interpreter needed to run the applications
- Source code is available as a script

Python Programming Language

- Interpreted Language
- General Purpose Language
- Open Source Language
- Portable language / Platform Independent / WORA
- Dynamicaly typed language (later)

Guido Van Rossum
89-91
C/C++/Java



Application that you write using python can be run on any of these

Python Interpreter

Win/Lin/Mac	Win/Lin/Mac	Win/Lin/Mac
64bit	32bit	16bit

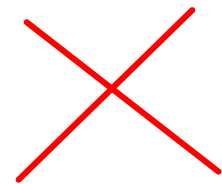
without modifications

Python Versions

2.x

3.x (latest)

2.x



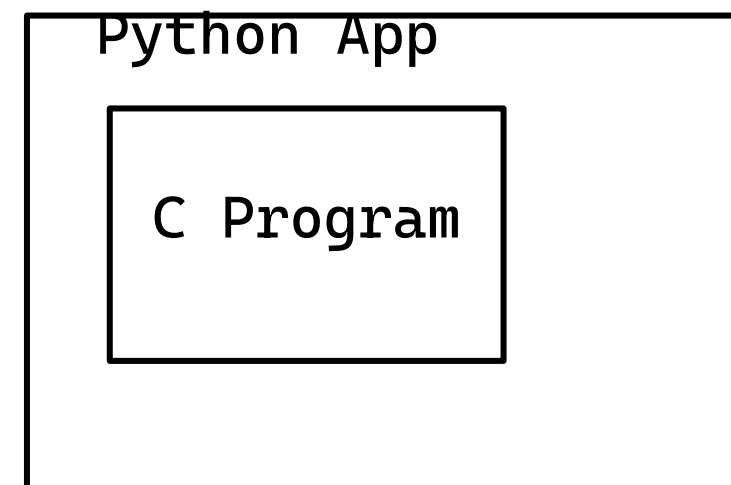
3.x

3.9.27 — 3.12.13 — 3.15.11

Python Interpreters

- Cython
- Jython
- RPython
- Pypi

Cython - C Python



Software needed to use / Develop python Programs

1. Python Interpreter
2. Text Editor – Notepad++, Atom, Sublime, VSCode, VIM, Geany, gedit, nano, emacs.....

Myth: Python needs big environments to develop & use

Interpreter+TextEditor+Misc tools (IDE) – Integrated Development Environment

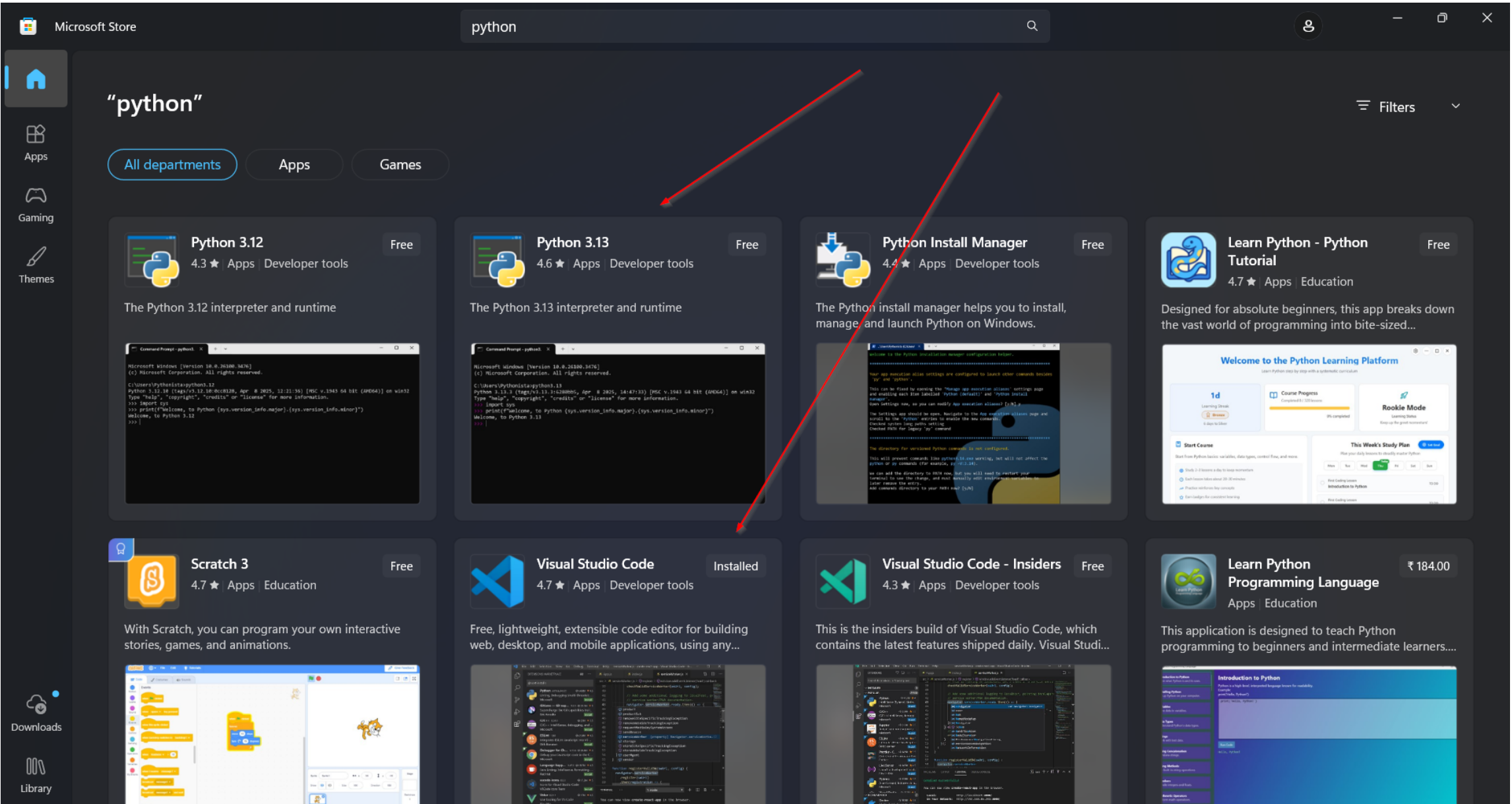
Pycharm, Spyder, jupyter Notebook, Conda env

recommendation:

If you are a beginner – Interpreter+VSCODE (without AI)

Installation of Softwares – Python, VSCODE

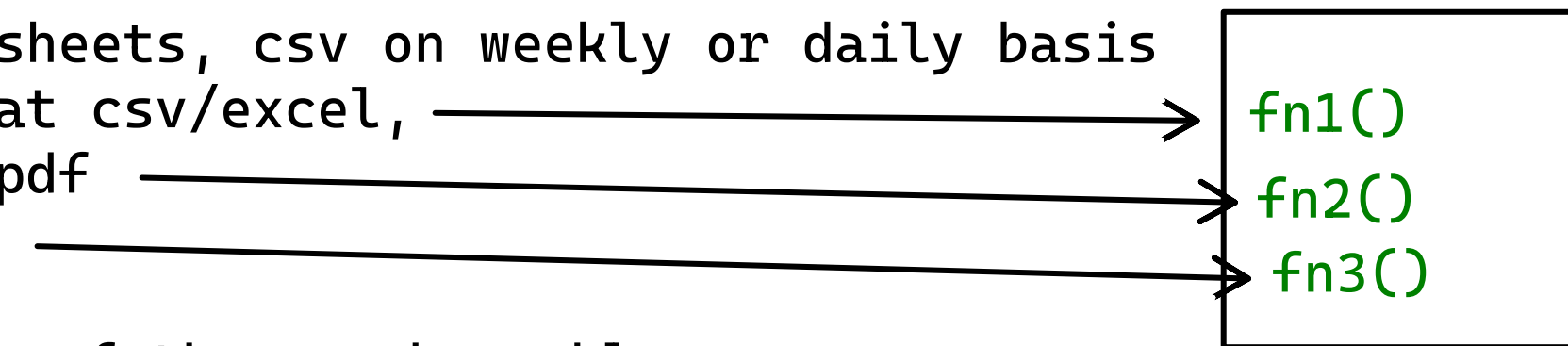
Lab: Installing Softwares



How to approach

You are a Consultant,
you get data in excel sheets, csv on weekly or daily basis
You have to process that csv/excel, _____
generate a reports in pdf _____
send it to management. _____

Single/Complete Solution



fn1()
fn2()
fn3()

Find Solutions for each of these sub problems

You have to process that csv/excel

Input: Excel/Csv

Output: Excel/Csv

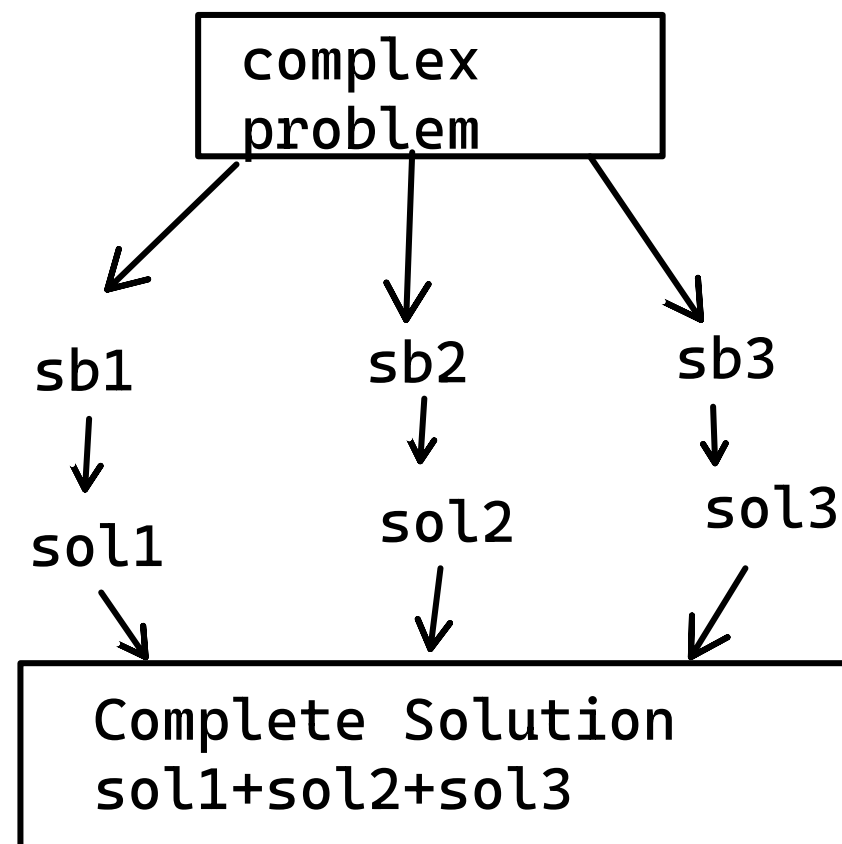
Logic: Cleaning of Data - Findout missing data & remove those
Data Format Correct
Date Format Correct

Programming in general

1. Why was that language created (Base paradigm) – Approach to problem
2. How to store Data Temporarily
3. How to Do Conditions
4. How to repeat (Loops / Functions)

Python Programming

1. Functional Approach – Solutions modeled in terms of Functions

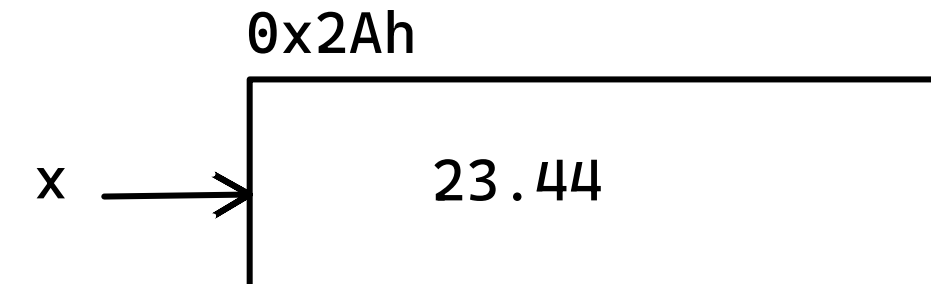


Python is a functional programming but also support Object oriented. Its an object based programming language(internal functionality is classes & objects)

2. How to store data temporarily(Variables)

Vishwas
23
23.45

x	=	23.44
label	Assignment	data
variable	operator	
identifier		



Dynamically Typed Language – You do not specify the data type before using a variable
Interpreter automatically detects it

Rules:

- alpha numeric & _ only
- Should not start with a number
- should not be a keyword

Conventions:

- Use meaningful names
- use snake_case

number_of_words

```

>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

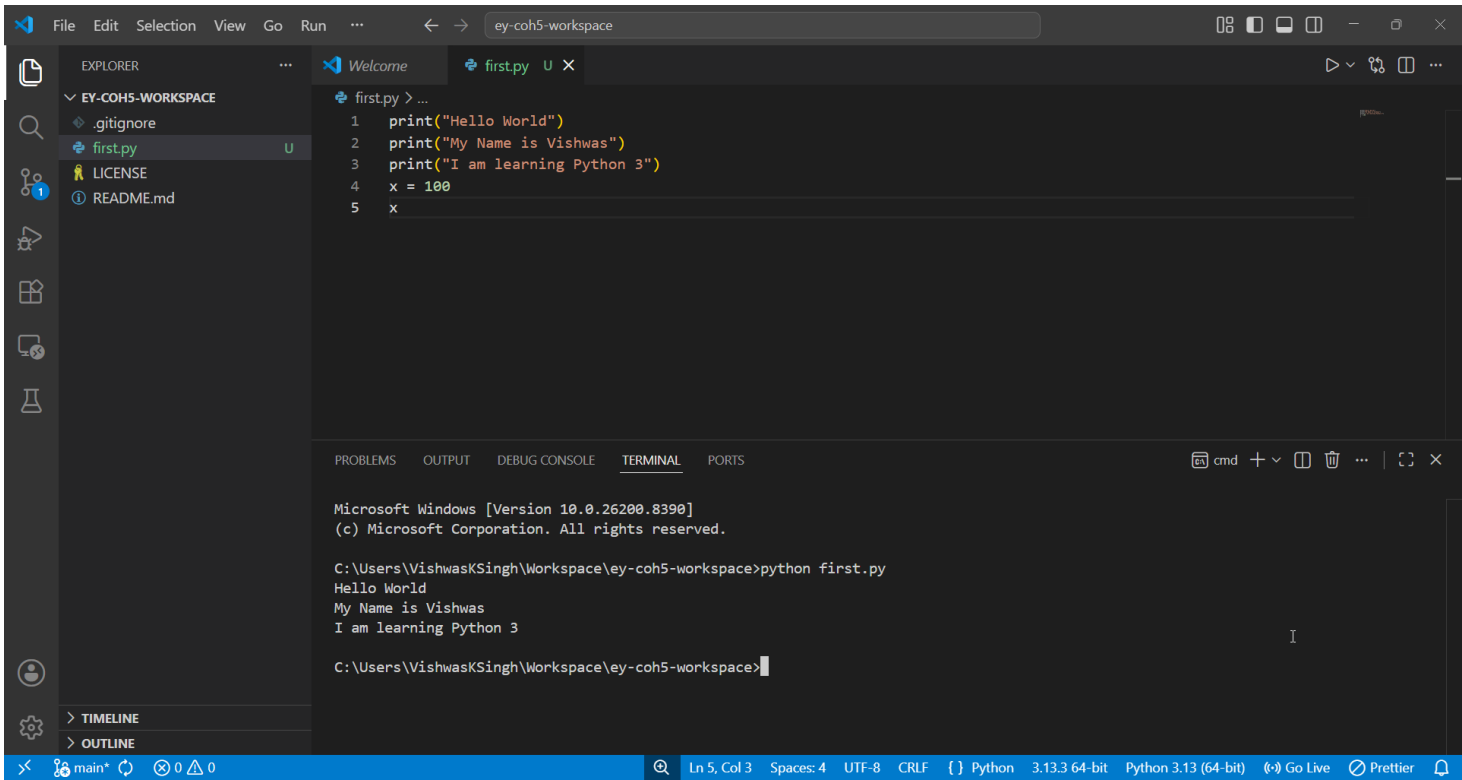
```

How to interact with Python

- Interactive mode

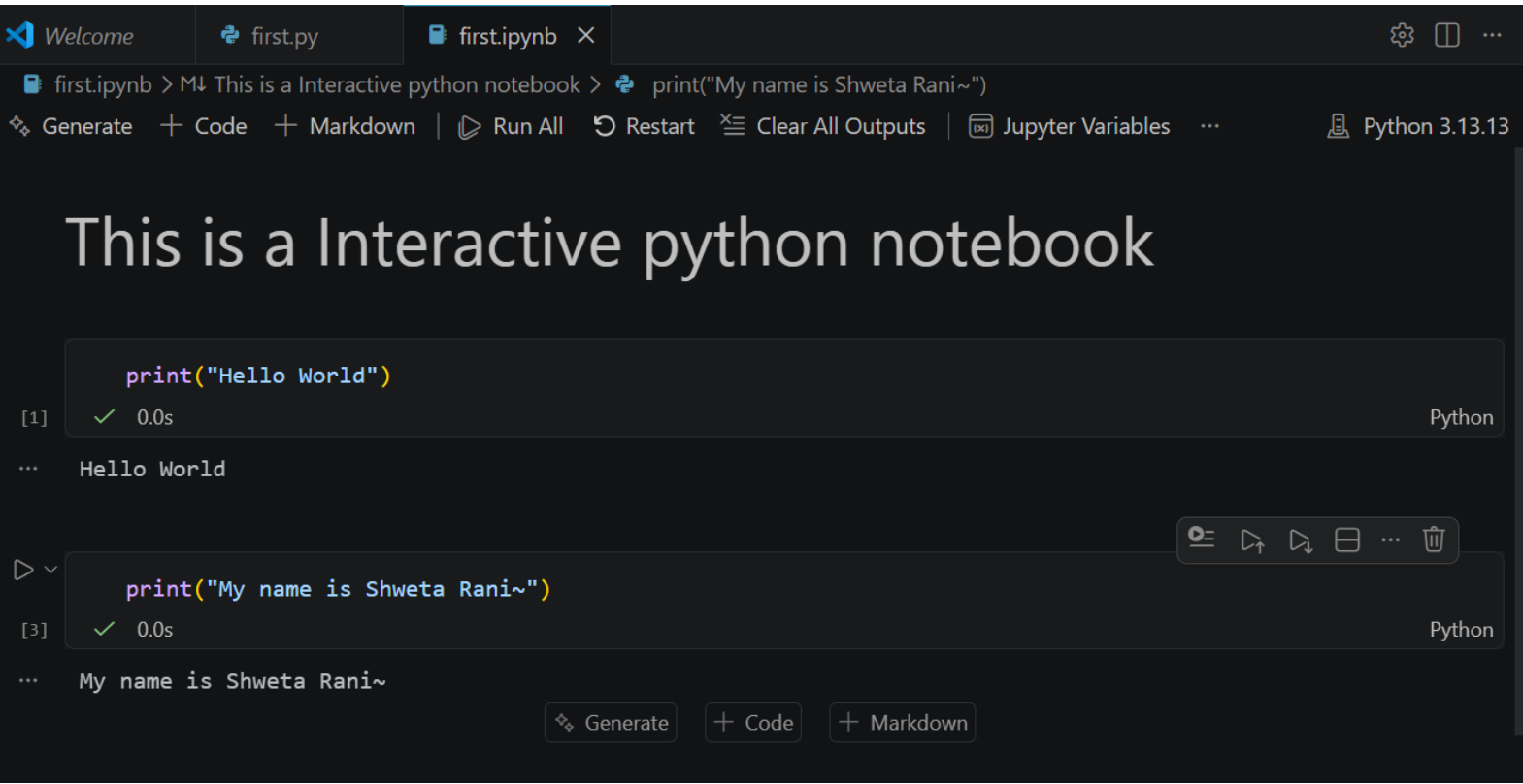
```
C:\Users\VishwasKSingh>python
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>> 2+3
5
>>> 4+5
9
>>> x = 10
>>> x
10
>>> quit()
```

- Script Mode



- working of snippets of code

- Notebook Mode



Data Types in Python – Numbers

```
>>> x = 100
>>> x1 = "100"
>>> type(x)
<class 'int'>
>>> type(x1)
<class 'str'>
>>> x3 = 23.45
>>> type(x3)
<class 'float'>
>>> x4 = 7812731278378173917927419273912387391283919712491287519274192485712948120381724912812032184102384124182491271249237424913712
>>> x4
7812731278378173917927419273912387391283919712491287519274192485712948120381724912812032184102384124182491271249237424913712
>>> x5 = 2123214234123.12312423141902749837498237498237498237498
>>> x5
2123214234123.123
>>> x6 = 12.12323345465765678798980978078567457564564
>>> x6
12.123233454657656
```

```
>>> x8 = 10_00_000
>>> x8
1000000
>>> x9 = 0b10101111
>>> x9
175
>>> type(x9)
<class 'int'>
>>> x10 = 0x1AB
>>> x10
427
>>> x11 = 0o123
>>> x11
83
>>> |
```

```
>>> l1 = 123.45e10
>>> l1
1234500000000.0
>>> 0.12345e4
1234.5
>>> l2 = complex(2,1)
>>> l2
(2+1j)
>>>
```

Strings

```
>>> name = "Vishwas K Singh"
>>> l6 = f"Hi my name is {name}"
>>> l6
'Hi my name is Vishwas K Singh'
>>> salary = 20000.456789
>>> l6 = f"Hi my name is {name}, My salary is {salary:.2f}"
>>> l6
'Hi my name is Vishwas K Singh, My salary is 20000.46'
>>> |
```


Taking Input from the user

- Whatever input you take from the user is always a string
- its the job of the developer to convert to right data type

```
>>> name = input("Enter your name: ")
Enter your name: vishwas k singh
>>> name
'vishwas k singh'
>>> num = input("Enter a number: ")
Enter a number: 34
>>> num
'34'
>>> type(num)
<class 'str'>
>>> num = int(num)
>>> num
34
>>> type(num)
<class 'int'>
>>> num = int(input("Enter a number: "))
Enter a number: 67
>>> type(num)
<class 'int'>
>>> num
67
>>>
```

```
2
3 name = input("Enter your name: ")
4 num1 = int(input("Enter first Number: "))
5 num2 = int(input("Enter second Number: "))
6 print(f"Welcome {name}, the sum of numbers entered is: {num1+num2}")
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

JUPYTER

```
C:\Users\VishwasKSingh\Workspace\ey-coh5-workspace>python 01-basics.py
Enter your name: vishwas
Enter first Number: 23
Enter second Number: 34
Welcome vishwas, the sum of numbers entered is: 57
```

1. 0 is always False. Non-Zero is always True

Short Circuit Evaluation:

- And Expression
out of 2 expressions (expr1 and expr2)
expr 2 is evaluated only when expr1 is True
- Or Expression
expr 2 is not evaluated at all when expr1 is already True

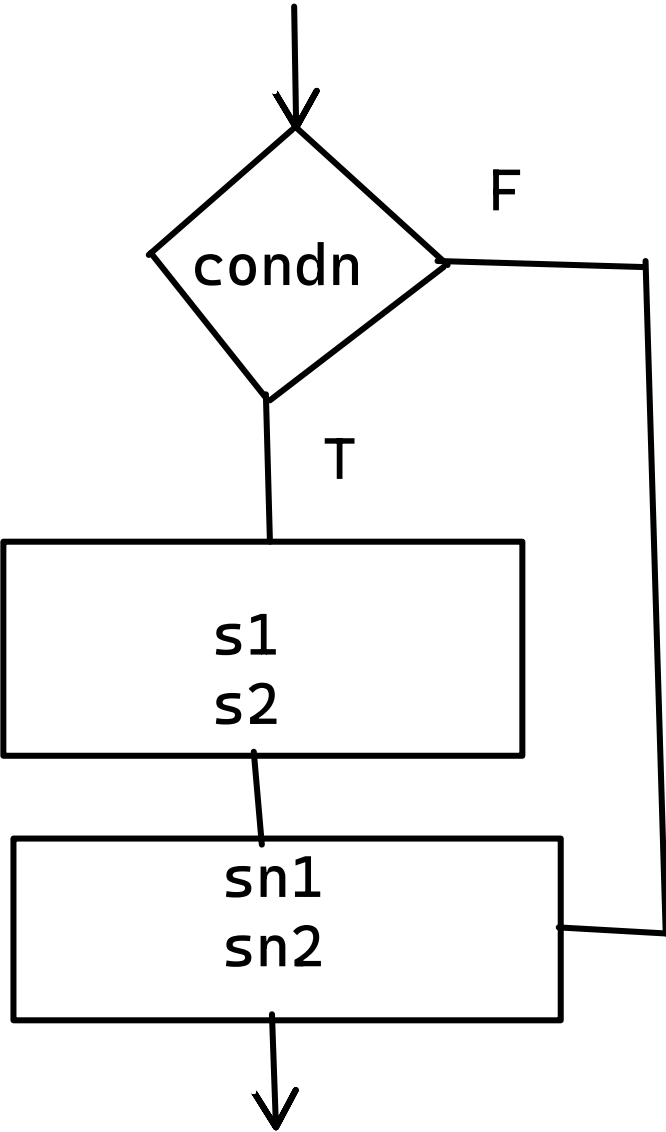
```
>>> 3 and 2
2
>>> 5 and 7
7
>>> 6 and 3%2
1
>>> 6 and 3/2
1.5
>>> 2 or 3
2
>>> 3/2 or 3
1.5
>>>
```

Guard Pattern

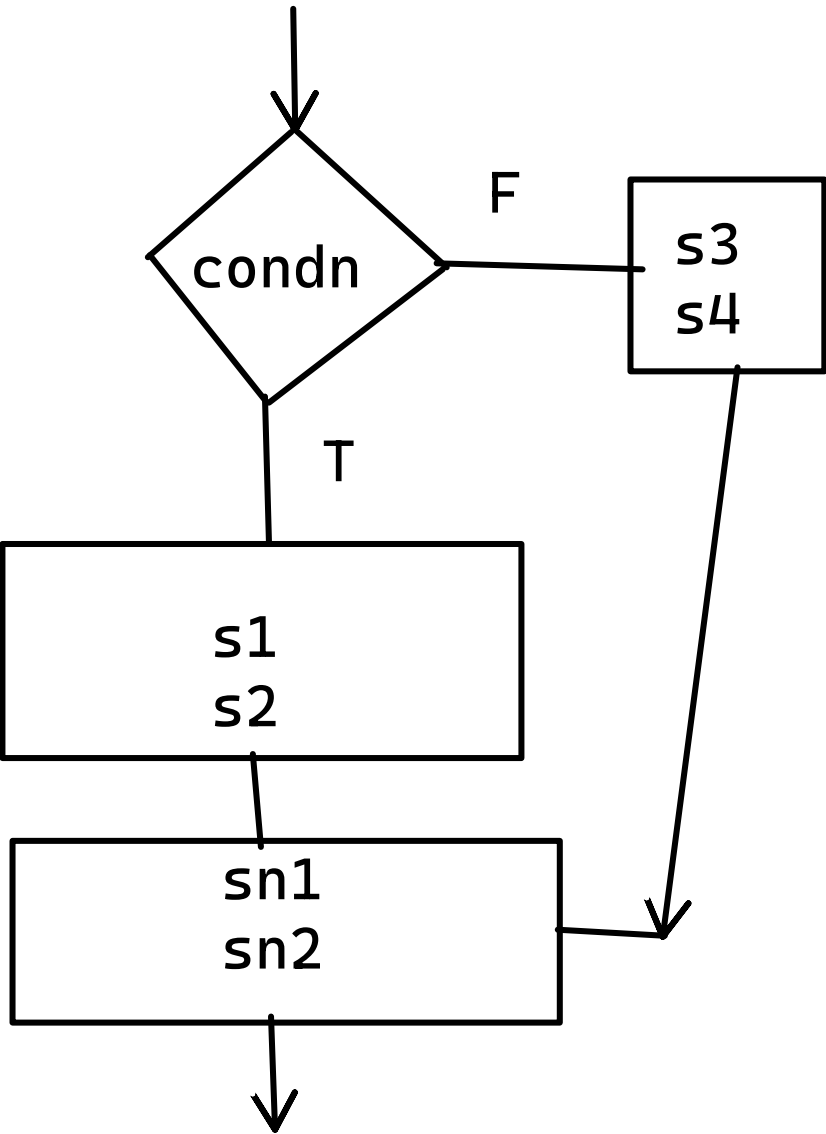
```
>>> x = 10
>>> y = 2
>>> x / y
5.0
>>> y = 0
>>> x / y
Traceback (most recent call last):
  File "<python-input-113>", line 1, in <module>
    x / y
    ~^~
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
5.0
>>> |
```

Conditional

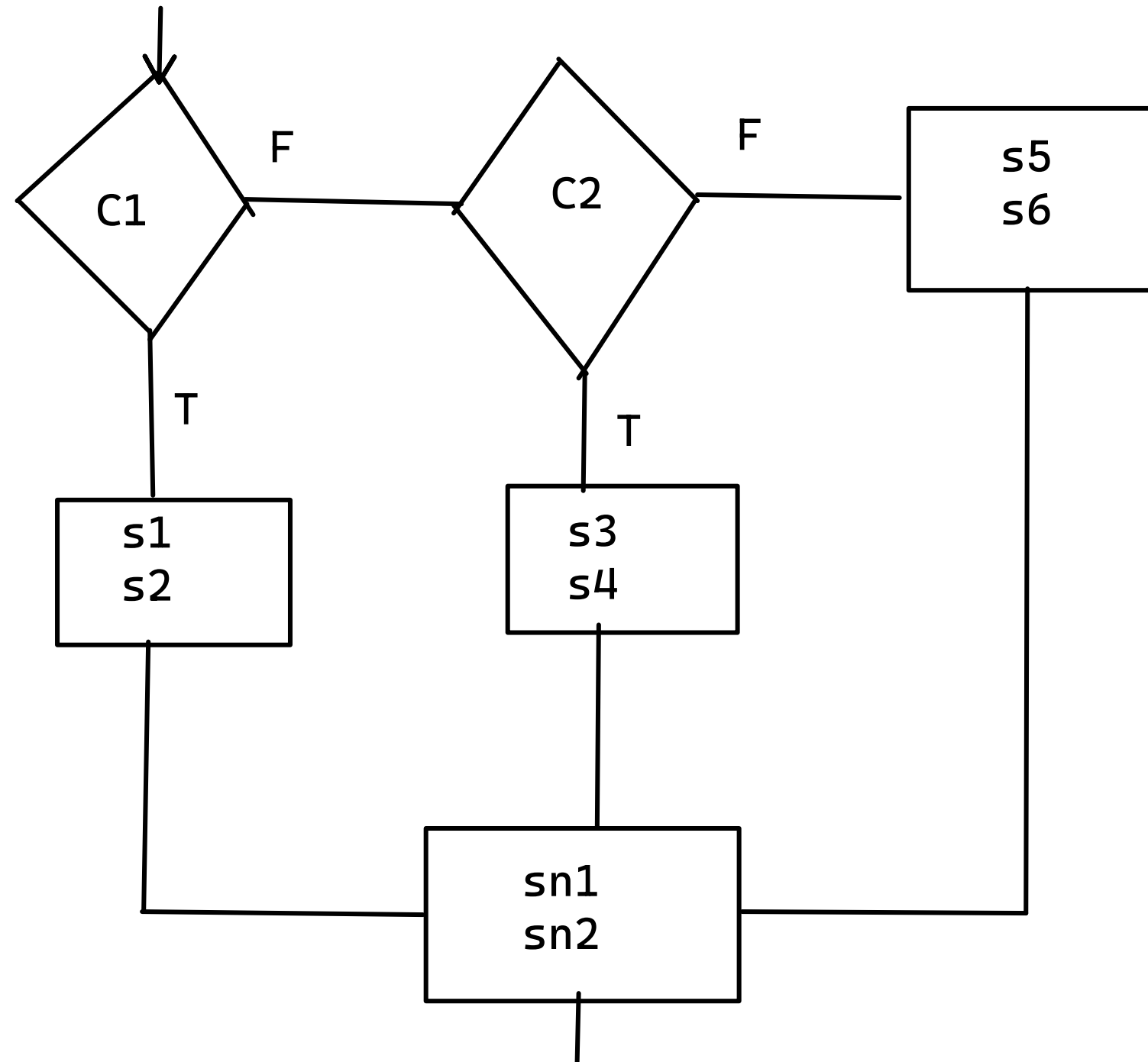
```
if condition:  
    s1  
    s2  
sn1  
sn2
```



```
if condn:  
    s1  
    s2  
else:  
    s3  
    s4  
sn1  
sn2
```




```
if condn1:  
    s1  
    s2  
elif condn2:  
    s3  
    s4  
else:  
    s5  
    s6  
sn1  
sn2
```



nested if

```
if condn:  
    if condn:  
        s1  
        s2  
    else:  
        s3  
        s4  
else:  
    s5  
    s6
```

Always note the order of condition check

Grade System

90 >	A
80 > < 90	B
70 > < 80	C
< 70	D

```
x = 75
if x > 50:
    print("x>50")
elif x > 30:
    print("x>30")
elif x > 70:
    print("x>70")
else:
    print("x<30")
```

```
marks = 80
if marks > 90:
    print("GA")
elif marks > 80 and marks < 90:
    print("GB")
elif marks > 70 and marks < 80:
    print("GC")
else:
    print("GD")
```

What all statements will be printed

`x>50`

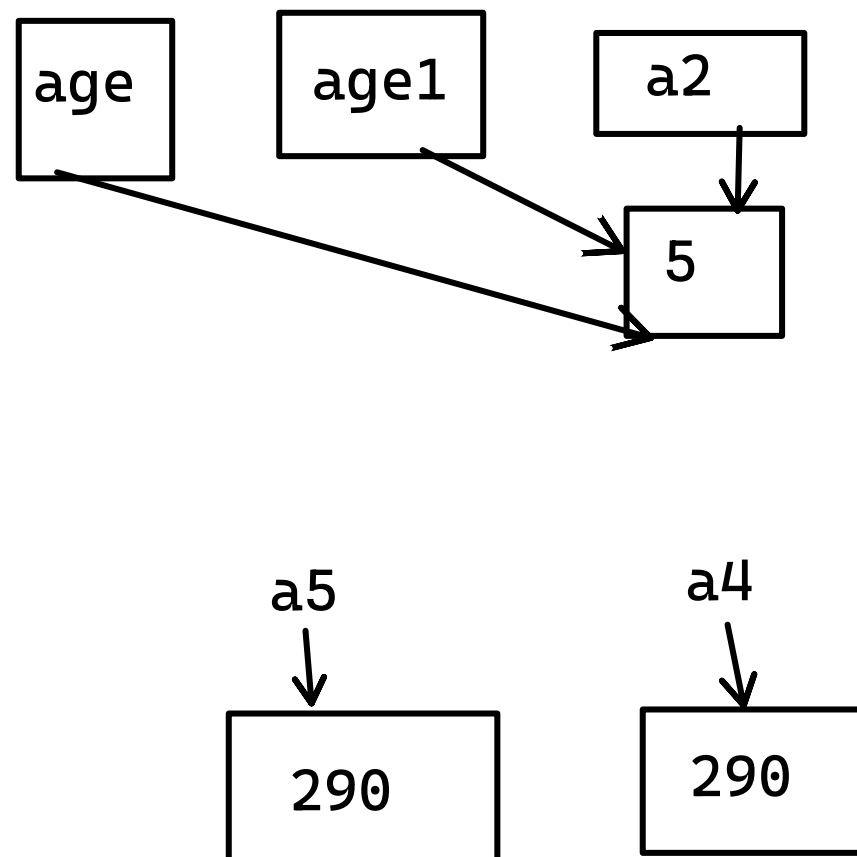
Print Statements

```
name = input("Enter second number: ")  
print(f"Welcome {name}, the sum of numbers entered is: {num1+num2}")  
print("Welcome", name, " the sum of numbers entered is: ", num1+num2)  
print("Welcome "+ name + " the sum of numbers entered is: " + str(num1+num2))
```

```
print("Hello", end="**")  
print("Vishwas", end="*")  
print("Singh")
```

```
>>> print("Hello"*3)  
HelloHelloHello
```

When you are storing integers
if the value is < 255 and multiple variables are storing the same value
then they will be pointed by same object



```
>>> age = 5
>>> id(age)
140722280866856
>>> age1 = 5
>>> id(age1)
140722280866856
>>> age2=250
>>> a3=250
>>> id(age2)
140722280874696
>>> id(a3)
140722280874696
>>> a4=290
>>> a5=290
>>> id(a4)
2709958056688
>>> id(a5)
2709958056016
>>>
```

```
>>> bin(23)
'0b10111'
>>> hex(23)
'0x17'
>>> oct(23)
'0o27'
>>> ord('a')
97
>>> chr(98)
'b'
```

```
>>> v1 = 0b1011
>>> v1
11
>>> type(v1)
<class 'int'>
>>> bin(v1)
'0b1011'
>>> v1
11
>>> print(v1)
11
>>> |
```

Python Programming Day 1

- What is Python
- Variable, Data Types, operators, Conditionals, Expressions

What to expect for tomorrow

- Loops, Data Structures - Lists, Strings, Dict, Tuples, Sets
- functions

Practice Problems:

1. Build a Temperature Converter
2. Build a Simple Interest Calculator
3. Brokerage Calculator
4. SIP vs Lumpsum Calculator (*)

Agenda

Loops

Strings

Lists

Dicts

Tuples

Sets

Functions

For

```
for i in range(start, stop, step):
    s1
    s2
```

Start - inclusive

Stop - Exclusive

range(1,10)

start - 1

stop - 9

While

```
init =
```

```
while condn:
```

```
    s1
```

```
    s2
```

```
    inc/dec
```

```
while True
```

```
while -200
```

```
while 100
```

```
break, continue
```

```
>>> x = 10
>>> while x > 0:
...     print(x)
...     x -=1
...
10
9
8
7
6
5
4
3
2
1
>>> x = 0
>>> while x <= 10:
...     print(x)
...     x += 1
...
0
1
2
3
4
5
6
7
8
9
10
>>>
```

```
>>> for i in range(10):
...     print(i)
...
0
1
2
3
4
5
6
7
8
9
>>> for i in range(1,10):
...     print(i)
...
1
2
3
4
5
6
7
8
9
>>>
```

```
>>> for i in range(20,1, -1):
...     print(i)
...
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
```

Scenario: Game

Take numbers from the user

1. User cannot enter a value less than 10, more than 100
2. You calculate the total & average of the numbers entered
3. stop when the sum is greater than 200 or average is more than 75
tell them in how many steps they achieved this.

when you know number of steps
or iterations goto for loop
else use while loop

Strings

- Immutable : Cannot be modified
- Sequence of characters

```
s1 = "vishwas"
s1[0]
s1[1]
```
- Indexing always from left to right except when step is -ve

```
>>> S1[::-1]
'HGNIS K SAWHSIV'
>>> S1[-9:-12:-1]
'SAW'
>>> S1[-3:-6:-1]
'NIS'
>>> S1[9:12:-1]
''
>>> S1[12:9:-1]
'NIS'
>>>
```

```
>>> S1[4:7]
'WAS'
>>> S1[-11:-8]
'WAS'
>>> S1[4:-8]
'WAS'
>>> S1[-11:7]
'WAS'
>>>
```

```
>>> S1 = "VISHWAS K SINGH"
>>> len(S1)
15
>>> S1[0]
'V'
>>> S1[1]
'I'
>>> S1[12]
'N'
>>>
```

```
>>> S1[:7]
'VISHWAS'
>>> S1[10:]
'SINGH'
>>> S1[:-8]
'VISHWAS'
>>> S1[-5:]
'SINGH'
>>> S1[:]
'VISHWAS K SINGH'
>>> S1[:2]
'VSWSKSNH'
>>> S1[:3]
'VHS N'
>>> S1[2:7:2]
'SWS'
>>> S1[-13:-8:2]
'SWS'
>>>
```

Lab: You are given a string, Validate if its email id, and extract the username if '@' and is only once → Valid
email: vishwas@cloudthat.com
username: vishwas

```
inp_str = input("Enter your email id: ")

if inp_str.find('@') == -1 or inp_str.count('@') > 1 :
    print("Invalid Email ID")
else:
    username = inp_str[: inp_str.find('@')]
    print(f"The user name for {inp_str} is {username}")
```

Lab: IPv4 Address

1. 4 Fields separated by ('.')
2. Each Field is an integer varying from 0 to 255

Write a program to Validate IPv4 Address

List

- Mutable
- Heterogeneous
- Dynamic

```
>>> l2 = list()
>>> type(l1)
<class 'list'>
>>> l3 = [23,45,67,89,"vishwas",[90,101], {1:"Mango",2:"Apple"}]
>>> l3[0]
23
>>> l3[2]
67
>>> l3[-1]
{1: 'Mango', 2: 'Apple'}
>>> l3[-2]
[90, 101]
>>> l3[:3]
[23, 45, 67]
>>> l3[:9]
[23, 45, 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}]
>>> l3[3:]
[89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}]
>>> l3[::-1]
[{1: 'Mango', 2: 'Apple'}, [90, 101], 'vishwas', 89, 67, 45, 23]
```

```
>>> l3
[23, 45, 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}]
>>> l1
[]
>>> l1.append("Mango")
>>> l1
['Mango']
>>> l1.append("Grapes")
>>> l1
['Mango', 'Grapes']
>>> l1.append("Papaya")
>>> l1
['Mango', 'Grapes', 'Papaya']
>>> l4 = l1.copy()
>>> l4
['Mango', 'Grapes', 'Papaya']
>>> l1
['Mango', 'Grapes', 'Papaya']
>>> l1.clear()
>>> l1
[]
>>> l4
['Mango', 'Grapes', 'Papaya']
>>>
```

```
>>> l1
[]
>>> l3
[23, 45, 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}]
>>> l4
['Mango', 'Grapes', 'Papaya']
>>> l4.append("Mango")
>>> l4
['Mango', 'Grapes', 'Papaya', 'Mango']
>>> l4.count("Mango")
2
>>> l3
[23, 45, 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}]
>>> l4
['Mango', 'Grapes', 'Papaya', 'Mango']
>>> l3.extend(l4)
>>> l3
[23, 45, 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya', 'Mango']
>>> l3.index("Mango")
7
>>> l3.insert(2, "Kiwi")
>>> l3
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya', 'Mango']
>>> l6
Traceback (most recent call last):
  File "<python-input-134>", line 1, in <module>
    l6
NameError: name 'l6' is not defined. Did you mean: 'l1'?
>>> l6 = l3.copy()
>>> l6
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya', 'Mango']
>>> l2
[]
>>> l2 = l3.copy()
>>> l2
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya', 'Mango']
>>>
```



```
>>> l6
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya', 'Mango']
>>> l6.pop()
'Mango'
>>> l6
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya']
>>> l6.remove([90,101])
>>> l6
[23, 45, 'Kiwi', 67, 89, 'vishwas', {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya']
>>> l3
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Mango', 'Grapes', 'Papaya', 'Mango']
>>> l3.remove('Mango')
>>> l3
[23, 45, 'Kiwi', 67, 89, 'vishwas', [90, 101], {1: 'Mango', 2: 'Apple'}, 'Grapes', 'Papaya', 'Mango']
>>> dir(list)
['_add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getat
tribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '
__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__',
'__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
>>> help(list.sort)
Help on method_descriptor:

sort(self, /, *, key=None, reverse=False) unbound builtins.list method
    Sort the list in ascending order and return None.

    The sort is in-place (i.e. the list itself is modified) and stable (i.e. the
    order of two equal elements is maintained).

    If a key function is given, apply it once to each list item and sort them,
    ascending or descending, according to their function values.

    The reverse flag can be set to sort in descending order.

>>> l7 = [23,45,1,2,100,9,1,4]
>>> l7.sort()
>>> l7
[1, 1, 2, 4, 9, 23, 45, 100]
>>> |
```

```
>>> for i in range(len(l7)):
...     print(i,l7[i])
...
0 1
1 1
2 2
3 4
4 9
5 23
6 45
7 100
>>> for i in l7:
...     print(i)
...
1
1
2
4
9
23
45
100
>>>
```

```
>>> l8
[93, 45, 1, 77, 96, 2, 1]
>>> l9
[93, 45, 1, 77, 96, 2, 1]
>>> l9.sort()
>>> l9
[1, 1, 2, 45, 77, 93, 96]
>>> l10
[93, 45, 1, 77, 96, 2, 1]
>>> l10.sort(reverse=True)
>>> l10
[96, 93, 77, 45, 2, 1, 1]
>>> l11 = l8.copyt()
Traceback (most recent call last):
  File "<python-input-171>", line 1, in <module>
    l11 = l8.copyt()
            ^^^^^^^
AttributeError: 'list' object has no attribute 'copyt'. Did you mean: 'copy'?
>>> l11 = l8.copy()
>>> l8
[93, 45, 1, 77, 96, 2, 1]
>>> l8.reverse()
>>> l8
[1, 2, 96, 77, 1, 45, 93]
>>> l11[::-1]
[1, 2, 96, 77, 1, 45, 93]
>>> |
```

Dict

- Key:value
- Key must be unique
- str/int (recommended as keys)

```
>>> d1 = {}
>>> type(d1)
<class 'dict'>
>>> d2 = dict()
>>> type(d2)
<class 'dict'>
>>> d3 = {"a":"apple","b":"Ball","c":"cat","d":"dog","e":"elephant"}
>>> d3 = {"a":"apple","b":"Ball","c":"cat","d":"dog","e":"elephant", 1:"apple",2:"grapes"}
>>> d3
{'a': 'apple', 'b': 'Ball', 'c': 'cat', 'd': 'dog', 'e': 'elephant', 1: 'apple', 2: 'grapes'}
>>> d3["b"]
'Ball'
>>> d3["c"]
'cat'
>>> d3["d"]
'dog'
>>> |
```



```
>>> d3
{'a': 'apple', 'b': 'Ball', 'c': 'cat', 'd': 'dog', 'e': 'elephant', 1: 'apple', 2: 'grapes', '*': [1, 'Vishwas', 2, 'Arjun']}
>>> d3[3]
Traceback (most recent call last):
  File "<python-input-199>", line 1, in <module>
    d3[3]
    ~~~~
KeyError: 3
>>> d3.get(3, "Not Available")
'Not Available'
>>> d3.get(2, "Not Available")
'grapes'
>>> dir(dict)
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
 '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__ne
w__', '__or__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook
__', 'clear', 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
>>> d3.keys()
dict_keys(['a', 'b', 'c', 'd', 'e', 1, 2, '*'])
>>> d3.values()
dict_values(['apple', 'Ball', 'cat', 'dog', 'elephant', 'apple', 'grapes', [1, 'Vishwas', 2, 'Arjun']])
>>> d3.items()
dict_items([('a', 'apple'), ('b', 'Ball'), ('c', 'cat'), ('d', 'dog'), ('e', 'elephant'), (1, 'apple'), (2, 'grapes'), (*, [1, 'Vishwas', 2, 'Arjun'])])
>>>
```

a list of blacklisted ips

ip: number of times found in log

ip: 0

ip_address: 5

```

>>> l1 = ["Mouse","Keyboard","Laptop","Monitor"]
>>> inventory = dict.fromkeys(l1,0)
>>> inventory
{'Mouse': 0, 'Keyboard': 0, 'Laptop': 0, 'Monitor': 0}
>>> inventory = dict.fromkeys(l1)
>>> inventory
{'Mouse': None, 'Keyboard': None, 'Laptop': None, 'Monitor': None}
>>> inventory = dict.fromkeys(l1,"Not Available")
>>> inventory
{'Mouse': 'Not Available', 'Keyboard': 'Not Available', 'Laptop': 'Not Available', 'Monitor': 'Not Available'}
>>> inventory.pop('Monitor')
'Not Available'
>>> inventory
{'Mouse': 'Not Available', 'Keyboard': 'Not Available', 'Laptop': 'Not Available'}
>>> inventory.pop('TV')
Traceback (most recent call last):
  File "<python-input-217>", line 1, in <module>
    inventory.pop('TV')
    ~~~~~^~~~~
KeyError: 'TV'
>>> inventory.pop('TV',"NA")
'NA'
>>> inventory.popitem()
('Laptop', 'Not Available')
>>> inventory
{'Mouse': 'Not Available', 'Keyboard': 'Not Available'}
>>> inventory.pop('Not Available')
Traceback (most recent call last):
  File "<python-input-221>", line 1, in <module>
    inventory.pop('Not Available')
    ~~~~~^~~~~
KeyError: 'Not Available'
>>> inventory
{'Mouse': 'Not Available', 'Keyboard': 'Not Available'}
>>> inventory.setdefault("Laptop","Not Available")
'Not Available'
>>> inventory
{'Mouse': 'Not Available', 'Keyboard': 'Not Available', 'Laptop': 'Not Available'}
>>> |

```

```

>>> inventory
{'Mouse': 'Not Available', 'Keyboard': 'Not Available', 'Laptop': 'Not Available'}
>>> l2 = [("Mouse", 200),("Laptop",99)]
>>> l2 = [("Mouse", 200),("Laptop",99),("Monitor",50)]
>>> inventory.update(l2)
>>> inventory
{'Mouse': 200, 'Keyboard': 'Not Available', 'Laptop': 99, 'Monitor': 50}
>>> l3 = [50,45,22,9]
>>> inventory.update(l3)
Traceback (most recent call last):
  File "<python-input-233>", line 1, in <module>
    inventory.update(l3)
    ~~~~~^~~~~
TypeError: cannot convert dictionary update sequence element #0 to a sequence
>>>

```

Tuple

- Immutable
- indexes, sliced
- just like list

```
>>> t1 = ("apple","mango",45,67,92)
>>> t1
('apple', 'mango', 45, 67, 92)
>>> t2 = "Vishwas","Arjun","Riya"
>>> t2
('Vishwas', 'Arjun', 'Riya')
>>> type(t2)
<class 'tuple'>
>>> t2[0]
'Vishwas'
>>> t2[-1]
'Riya'
>>> t2[:3]
('Vishwas', 'Arjun', 'Riya')
>>> t2[2:]
('Riya',)
>>>
```

```
>>> t3 = a,b,c
>>> t3
('Banana', 'Grapes', 'Papaya')
>>> x,y,z = t3
>>> x
'Banana'
>>> y
'Grapes'
>>> z
'Papaya'
>>>
```

```
>>> t4 = 1,2,5,7
>>> t5 = 1,2,9,6
>>> t4 > t5
False
>>> t4 , t5
((1, 2, 5, 7), (1, 2, 9, 6))
>>> t4 < t5
True
>>> |
```

Set

- Sequence: Stores distinct Values

```
>>> s1 = {23,45,67,1,2,1,67,23}
>>> s1
{1, 2, 67, 23, 45}
>>> dir(set)
['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__',
__getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__',
__iter__', '__le__', '__len__', '__lshift__', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__',
__rsub__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update',
'discard', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update',
'update']
>>> s1
{1, 2, 67, 23, 45}
>>> s1.add(99)
>>> s1
{1, 2, 67, 99, 23, 45}
>>> s1
{1, 2, 67, 99, 23, 45}
>>> s1.pop()
1
>>> help(set.add)
Help on method_descriptor:

add(self, object, /) unbound builtins.set method
    Add an element to a set.

    This has no effect if the element is already present.

>>> help(set.pop)
Help on method_descriptor:

pop(self, /) unbound builtins.set method
    Remove and return an arbitrary set element.

    Raises KeyError if the set is empty.

>>> s1.pop()
2
>>> s1.pop()
67
>>> |
```

Log Analysis & Threat Detection

Assume, we have raw data

1. Parse the log Entries - str
2. Maintain ordered sequence of events - lists
3. Ensure that IP addresses in system are unique - sets
4. Keep the log metadata immutable - tuples
5. Map specific addresses to failed login counts - dict

```
blacklisted_ips = {"192.168.1.50", "10.0.0.99", "185.220.101.5"}
raw_logs = [
    "2026-06-02 10:01:22 - IP: 192.168.1.20 - Action: LOGIN_SUCCESS",
    "2026-06-02 10:02:05 - IP: 192.168.1.50 - Action: LOGIN_FAILED",
    "2026-06-02 10:03:11 - IP: 172.16.0.5 - Action: LOGIN_FAILED",
    "2026-06-02 10:03:45 - IP: 172.16.0.5 - Action: LOGIN_FAILED",
    "2026-06-02 10:04:12 - IP: 172.16.0.5 - Action: LOGIN_FAILED",
    "2026-06-02 10:05:00 - IP: 192.168.1.20 - Action: LOGIN_SUCCESS"
]
```


Summary:

1. loops
2. Data Structures – List, Dict, Tuples, Sets & Strings

Expectation for Tomorrow:

functions

modules packages

OOPs

File Handling & Exception Handling

Practice Problems:

- | | |
|--|--|
| 1. Frequency Counter
vishwas
'v':1, 'i':1, 's':2..... | 3. Brokerage Calculator v2
Brokerage for any of asset class |
| 2. Numbers to words converter
123
One Hundred Twenty Three | 4. Loan Calculator v2 for various combination of principal, years & rate |
| | 5. SIP/Lumpsum vs → how money grows year-on-year |